

SE446: Big Data Engineering

Week 6B: Advanced HiveQL — NYC Taxi Analytics

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 Recap & New Dataset
- 2 ROLLUP & CUBE
- 3 Window Functions
- 4 JOINS in Hive
- 5 EXPLAIN Plans & Optimization
- 6 ORC vs TextFile
- 7 Summary

Recap: What We Know About Hive

Week 5 — Hive Fundamentals:

- ✓ CREATE TABLE, LOAD DATA
- ✓ SELECT, WHERE, GROUP BY
- ✓ ORDER BY, COUNT, AVG
- ✓ Managed vs. External tables
- ✓ Schema-on-Read

Today's Goal — Go Deeper:

- + ROLLUP and CUBE
- + Window functions
- + Multi-table JOINS
- + EXPLAIN plans
- + ORC vs TextFile

Why These Features?

Real-world analytics needs **subtotals**, **rankings**, and **lag/lead comparisons** — none of which basic GROUP BY can do.

NYC Yellow Taxi Dataset

A Classic Big Data Dataset

Millions of taxi trips recorded in New York City. We use a sample of **10,000 rows** from Jan–Mar 2024.

Key Columns:

Column	Type	Description
pickup_datetime	TIMESTAMP	Trip start time
dropoff_datetime	TIMESTAMP	Trip end time
passenger_count	INT	Number of passengers
trip_distance	DOUBLE	Miles traveled
fare_amount	DOUBLE	Base fare (\$)
tip_amount	DOUBLE	Tip (\$)
total_amount	DOUBLE	Total charge (\$)
payment_type	INT	1=Credit, 2=Cash, 3=No charge
pu_location_id	INT	Pickup zone (1–265)
do_location_id	INT	Dropoff zone (1–265)

Creating the Table

```
CREATE EXTERNAL TABLE IF NOT EXISTS nyc_taxi (  
    vendor_id          INT,  
    pickup_datetime    TIMESTAMP,  
    dropoff_datetime   TIMESTAMP,  
    passenger_count    INT,  
    trip_distance      DOUBLE,  
    pu_location_id     INT,  
    do_location_id     INT,  
    payment_type       INT,  
    fare_amount        DOUBLE,  
    tip_amount         DOUBLE,  
    tolls_amount       DOUBLE,  
    total_amount       DOUBLE  
)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/data/nyc_taxi/';
```

```
-- Quick check  
SELECT COUNT(*) AS total_trips FROM nyc_taxi;  
-- 10000
```

The Subtotal Problem

Business Question

*"Show me average fare by payment type **and** month, **plus** a subtotal per payment type, **plus** a grand total."*

Basic GROUP BY can't do this:

- GROUP BY payment_type, month — detail rows only
- GROUP BY payment_type — subtotals only
- SELECT AVG(fare) — grand total only
- You'd need UNION ALL of all three queries!

Solution

ROLLUP and CUBE generate **all grouping levels** in a single query.

GROUP BY ROLLUP

ROLLUP(A, B) produces: (A, B) + (A) + (grand total)

```
SELECT
    payment_type,
    MONTH(pickup_datetime) AS trip_month,
    ROUND(AVG(fare_amount), 2) AS avg_fare,
    COUNT(*) AS trip_count
FROM nyc_taxi
GROUP BY payment_type, MONTH(pickup_datetime)
WITH ROLLUP;
```

Result includes 3 levels:

payment_type	trip_month	avg_fare	trip_count
1	1	15.30	3200
1	2	14.80	2900
1	NULL	15.10	6100
2	1	12.50	1800
2	NULL	12.80	3700
NULL	NULL	14.20	10000

Green = subtotal per payment_type Yellow = grand total

GROUP BY CUBE

CUBE(A, B) produces **all possible combinations**: (A, B) + (A) + (B) + (grand total)

```
SELECT
    payment_type,
    MONTH(pickup_datetime) AS trip_month,
    ROUND(AVG(fare_amount), 2) AS avg_fare,
    COUNT(*) AS trip_count
FROM nyc_taxi
GROUP BY payment_type, MONTH(pickup_datetime)
WITH CUBE;
```

Extra rows compared to ROLLUP:

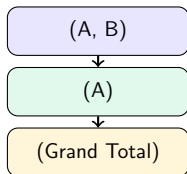
payment_type	trip_month	Meaning
1	1	Credit card trips in January
1	NULL	All credit card trips (subtotal)
NULL	1	All payment types in January
NULL	NULL	Grand total

ROLLUP vs CUBE

ROLLUP: hierarchical subtotals (left-to-right). CUBE: **all** dimension combinations (2^n groupings for n columns).

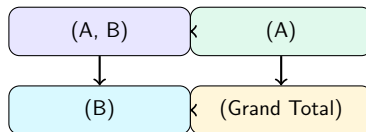
ROLLUP vs CUBE — Visual

ROLLUP(A, B)



3 grouping levels

CUBE(A, B)



$2^2 = 4$ grouping levels

General formula:

- ROLLUP on n columns $\rightarrow n + 1$ grouping levels
- CUBE on n columns $\rightarrow 2^n$ grouping levels

What Are Window Functions?

Definition

Window functions perform calculations **across a set of rows related to the current row** — without collapsing the result into fewer rows like GROUP BY.

GROUP BY = Aggregation

- 10 rows → 3 groups → **3 rows**
- Loses individual row detail

Window Functions

- 10 rows → still **10 rows**
- Each row keeps its detail + a computed value

General Syntax: `function() OVER ( [PARTITION BY ...] [ORDER BY ...] )`

RANK, DENSE_RANK, ROW_NUMBER

Goal: Rank pickup locations by total revenue.

```
SELECT
  pu_location_id,
  ROUND(SUM(total_amount), 2) AS total_revenue,
  RANK() OVER (ORDER BY SUM(total_amount) DESC)
    AS rank,
  DENSE_RANK() OVER (ORDER BY SUM(total_amount) DESC)
    AS dense_rank,
  ROW_NUMBER() OVER (ORDER BY SUM(total_amount) DESC)
    AS row_num
FROM nyc_taxi
GROUP BY pu_location_id
LIMIT 5;
```

Location	Revenue	RANK	DENSE_RANK	ROW_NUM
161	45,200	1	1	1
237	38,900	2	2	2
236	38,900	2	2	3
170	32,100	4	3	4

Key Difference on Ties

RANK skips (1, 2, 2, 4). DENSE_RANK doesn't skip (1, 2, 2, 3). ROW_NUMBER is always unique.

RANK with PARTITION BY

Goal: Top-3 pickup locations **per** payment type.

```
SELECT * FROM (  
  SELECT  
    payment_type,  
    pu_location_id,  
    ROUND(SUM(total_amount), 2) AS revenue,  
    RANK() OVER (  
      PARTITION BY payment_type  
      ORDER BY SUM(total_amount) DESC  
    ) AS rnk  
  FROM nyc_taxi  
  GROUP BY payment_type, pu_location_id  
) ranked  
WHERE rnk <= 3;
```

How PARTITION BY Works

PARTITION BY payment_type creates a **separate window** for each payment type — ranking restarts at 1 within each group.

LAG and LEAD — Comparing Adjacent Rows

Definition

LAG(col, n) returns the value from *n* rows **before**.

LEAD(col, n) returns the value from *n* rows **after**.

Goal: Compare each day's average fare to the previous day.

```
SELECT
    TO_DATE(pickup_datetime)      AS trip_date,
    ROUND(AVG(fare_amount), 2)    AS avg_fare,
    ROUND(LAG(AVG(fare_amount), 1)
           OVER (ORDER BY TO_DATE(pickup_datetime)), 2)
           AS prev_day_fare,
    ROUND(AVG(fare_amount) -
          LAG(AVG(fare_amount), 1)
           OVER (ORDER BY TO_DATE(pickup_datetime)), 2)
           AS fare_diff
FROM nyc_taxi
GROUP BY TO_DATE(pickup_datetime)
ORDER BY trip_date
LIMIT 5;
```

Common Pattern

value - LAG(value) gives the **change from previous period** — essential for trend analysis, growth calculations, and anomaly detection.

Running Totals: SUM OVER

Goal: Compute cumulative revenue by date.

```
SELECT
  TO_DATE(pickup_datetime) AS trip_date,
  ROUND(SUM(total_amount), 2) AS daily_revenue,
  ROUND(SUM(SUM(total_amount)) OVER (
    ORDER BY TO_DATE(pickup_datetime)
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ), 2) AS running_total
FROM nyc_taxi
GROUP BY TO_DATE(pickup_datetime)
ORDER BY trip_date;
```

trip_date	daily_revenue	running_total
2024-01-01	5,200	5,200
2024-01-02	6,100	11,300
2024-01-03	4,800	16,100
2024-01-04	7,200	23,300

Frame Clause

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

= Sum from the first row up to the current row.

NTILE — Dividing Into Groups

Goal: Divide trips into 4 fare quartiles.

```
SELECT
    fare_amount,
    trip_distance,
    NTILE(4) OVER (ORDER BY fare_amount) AS fare_quartile
FROM nyc_taxi
LIMIT 10;
```

fare_amount	trip_distance	fare_quartile
3.50	0.8	1
5.00	1.2	1
12.00	3.1	2
18.50	5.0	3
45.00	12.5	4

Use Case

NTILE(4) divides N rows into 4 approximately equal groups.

If $N = 10,000$, each quartile = $\lfloor 10,000/4 \rfloor = 2,500$ rows.

Window Functions Summary

Function	What It Does	Use Case
RANK()	Rank with gaps on ties	Top-N with ties
DENSE_RANK()	Rank without gaps	Continuous ranking
ROW_NUMBER()	Unique sequential number	Deduplication
LAG(col, n)	Value from n rows before	Day-over-day change
LEAD(col, n)	Value from n rows after	Forward comparison
SUM() OVER	Running/cumulative total	Growth tracking
NTILE(k)	Divide into k equal groups	Quartiles/percentiles

Key Syntax Pattern

```
function() OVER (PARTITION BY group_col ORDER BY sort_col)
```

PARTITION BY is optional — without it, the window spans all rows.

Creating a Lookup Table for Zones

Problem: Location IDs (1–265) are meaningless. We need a lookup table.

```
CREATE EXTERNAL TABLE IF NOT EXISTS taxi_zones (  
    location_id INT,  
    borough STRING,  
    zone_name STRING,  
    service_zone STRING  
)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
LOCATION '/data/taxi_zones/';
```

Sample rows:

ID	Borough	Zone	Service
161	Manhattan	Midtown Center	Yellow Zone
237	Manhattan	Upper East Side South	Yellow Zone
132	Queens	JFK Airport	Airports

JOIN: Revenue by Borough

Goal: Total revenue per pickup borough.

```
SELECT
  z.borough,
  COUNT(*)           AS trip_count,
  ROUND(SUM(t.total_amount), 2) AS total_revenue,
  ROUND(AVG(t.fare_amount), 2) AS avg_fare
FROM nyc_taxi t
JOIN taxi_zones z
  ON t.pu_location_id = z.location_id
GROUP BY z.borough
ORDER BY total_revenue DESC;
```

Borough	Trips	Revenue	Avg Fare
Manhattan	7,200	120,500	14.50
Queens	1,500	35,200	18.90
Brooklyn	800	12,100	12.30
Bronx	300	3,800	10.50

Hive JOIN Notes

Hive converts JOINs to MapReduce/Tez tasks. The **smaller table** should be on the LEFT for Map-Side Join optimization. Use `/*+ MAPJOIN(z) */` hint.

Self-JOIN Patterns

Goal: Find routes where average fare significantly differs by direction.

```
SELECT
  a.pu_location_id AS from_zone,
  a.do_location_id AS to_zone,
  a.avg_fare AS fare_A_to_B,
  b.avg_fare AS fare_B_to_A,
  ROUND(a.avg_fare - b.avg_fare, 2) AS fare_diff
FROM (
  SELECT pu_location_id, do_location_id,
         ROUND(AVG(fare_amount),2) AS avg_fare
  FROM nyc_taxi GROUP BY pu_location_id, do_location_id
) a
JOIN (
  SELECT pu_location_id, do_location_id,
         ROUND(AVG(fare_amount),2) AS avg_fare
  FROM nyc_taxi GROUP BY pu_location_id, do_location_id
) b
ON a.pu_location_id = b.do_location_id
   AND a.do_location_id = b.pu_location_id
WHERE ABS(a.avg_fare - b.avg_fare) > 5
LIMIT 5;
```

EXPLAIN: Understanding Query Execution

What Is EXPLAIN?

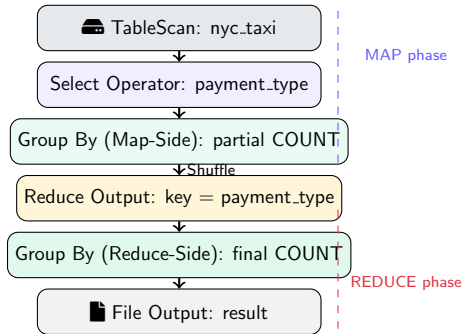
EXPLAIN shows the **execution plan** Hive will use *before* running the query. Essential for understanding performance.

```
EXPLAIN
SELECT payment_type, COUNT(*)
FROM nyc_taxi
GROUP BY payment_type;
```

Key elements to look for:

- **Stage dependencies:** how many MapReduce/Tez stages
- **TableScan:** which table is being scanned
- **Filter Operator:** whether predicates are pushed down
- **Group By Operator:** aggregation strategy (hash vs. sort)
- **Reduce Output Operator:** shuffle/sort keys
- **File Output Operator:** result format and location

Reading an EXPLAIN Plan



File Formats: TextFile vs ORC

TextFile (CSV)

- Human-readable
- No compression by default
- Full table scan for every query
- Suitable for loading/debugging

```
STORED AS TEXTFILE
```

ORC (Optimized Row Columnar)

- Columnar format
- Built-in compression (Zlib/Snappy)
- Predicate pushdown (skips irrelevant stripes)
- 75%+ smaller files

```
STORED AS ORC  
TBLPROPERTIES ("orc.compress"="ZLIB")
```

Rule of Thumb

Use **TextFile** for raw data landing. Convert to **ORC** for analytics. ORC queries run **2–10× faster** on the same data.

Converting to ORC

```
-- Step 1: Create ORC table (same schema)
CREATE TABLE nyc_taxi_orc
STORED AS ORC
TBLPROPERTIES ("orc.compress" = "ZLIB")
AS SELECT * FROM nyc_taxi;

-- Step 2: Compare file sizes
-- hdfs dfs -du -s -h /user/hive/warehouse/nyc_taxi
-- hdfs dfs -du -s -h /user/hive/warehouse/nyc_taxi_orc
```

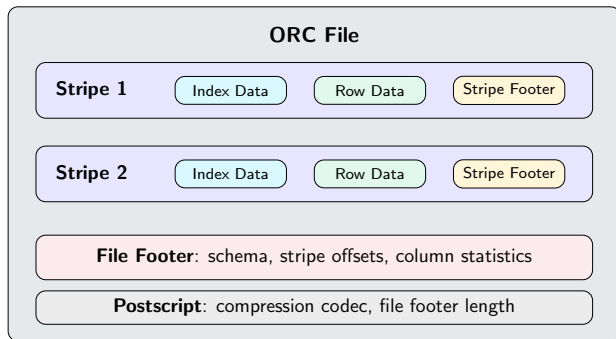
Typical results on our 10K-row dataset:

Format	Size	Query Time	Speedup
TextFile	~4.5 MB	25 sec	—
ORC (ZLIB)	~0.8 MB	8 sec	3.1×

Lab Exercise

You will run the same GROUP BY query on both tables and compare execution time and bytes read.

ORC Internal Structure



Predicate Pushdown

Stripe Index Data includes **min/max values** per column. If `WHERE fare > 100`, stripes where `max(fare) ≤ 100` are **skipped entirely**.

Key Takeaways

- 1 **ROLLUP**: hierarchical subtotals ($n + 1$ levels)
- 2 **CUBE**: all dimension combinations (2^n levels)
- 3 **Window Functions**: compute values *across rows* without collapsing
 - RANK/DENSE_RANK/ROW_NUMBER for rankings
 - LAG/LEAD for row-over-row comparison
 - SUM() OVER for running totals
 - NTILE(k) for equal-size buckets
- 4 **JOINS**: enrich data with lookup tables (taxi_zones)
- 5 **EXPLAIN**: read execution plans before running queries
- 6 **ORC**: columnar + compressed = 2–10× faster than TextFile



Remember

Basic Hive	SELECT, WHERE, GROUP BY
Advanced Hive	ROLLUP, CUBE, Window Functions, JOINS
Optimization	ORC format + EXPLAIN plan analysis

Next Session: Apache Spark

Week 7A: Spark Fundamentals — RDDs and Transformations

- Why Spark? — In-memory processing vs. MapReduce disk I/O
- Spark architecture: Driver, Executors, SparkContext
- RDDs: Resilient Distributed Datasets
- Transformations (lazy) vs. Actions (trigger execution)
- Crime analysis rewritten in PySpark

Pre-Class Preparation

- Review Python basics (lists, lambdas, tuples)
- SSH into cluster: `ssh root@134.209.172.50`
- Test PySpark: `pyspark --master yarn`

Midterm 1 Scope

Midterm 1 covers Weeks 1–6

- **Week 1–2:** HDFS architecture, commands, replication
- **Week 3–4:** MapReduce model, Word Count, Combiners, Partitioners
- **Week 5:** Hive fundamentals — DDL, DML, schema-on-read
- **Week 6A:** YARN — architecture, lifecycle, schedulers, CLI
- **Week 6B:** Advanced HiveQL — ROLLUP, CUBE, window functions, JOINS, ORC

Exam Format

MCQ + Numerical + Short Answer. Practice interpreting YARN Web UI screenshots, writing window function queries, and reading EXPLAIN plans.